

The OMNeT++ simulation framework

Performance Evaluation of Computer Systems and Networks

MSc in Computer Engineering

MSc in Artificial Intelligence and Data Engineering

Academic Year 2026/2027

Dr. Ing. Giovanni Nardini



UNIVERSITÀ DI PISA

OMNeT++ basics

The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

OMNeT++


<http://omnetpp.org>

It is a simulation framework!

OMNeT++
Discrete Event Simulator

OMNeT++ is an extensible, modular, component-based C++ simulation library and framework, primarily for building network simulators.

Featured Projects



Modular & object oriented	Eclipse-based IDE	Open source
Discrete event-based	Powerful GUI and command line interface	Extensions available

The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

3

Discrete-event simulation **framework**

It provides users with **basic building blocks** to define a simulation: event queue, statistic collection, random number generation, etc. Moreover, it provides a complete **simulation library**, i.e. classes, functions, data structures that help you building your simulation model.

It provides a number of **tools** that ease the development and maintenance of the code, like integrated debugging and validation testing, as well as statistics generation and automated management of simulation campaigns.

The OMNeT++ ecosystem includes many **libraries**, often interoperable (e.g., Simu5G + Veins for 5G-connected vehicles)

- Free for academic use
- Used by many top ICT companies
- Very large active community worldwide

OMNeT++ philosophy

From the very first lines of the OMNeT++ manual:

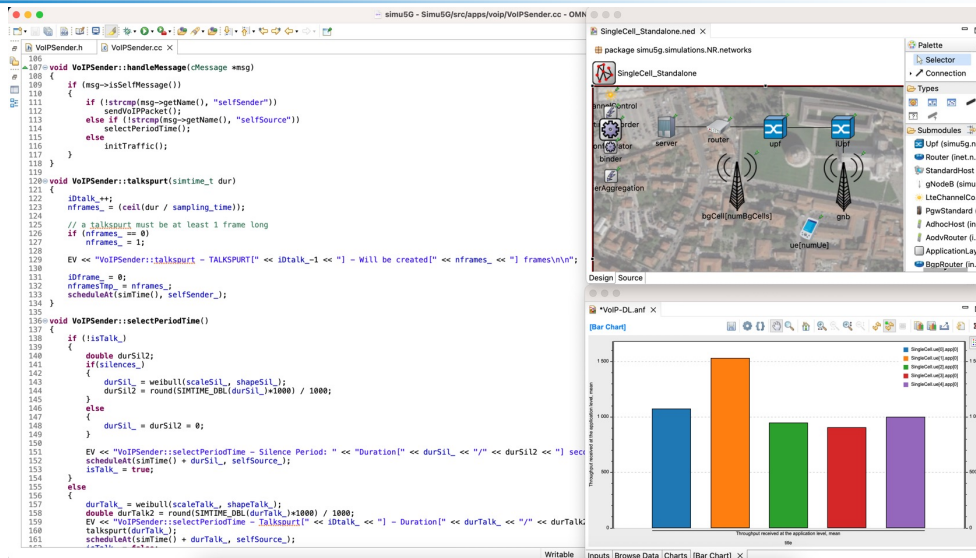
“OMNeT++ itself is *not* a simulator of anything concrete, but rather **provides infrastructure and tools for writing simulations.**

One of the fundamental ingredients of this infrastructure is a component architecture for simulation models. Models are assembled from reusable components termed **modules.**

Well-written modules are truly reusable, and can be combined in various ways **like LEGO blocks”**

OMNeT++ mandates a **modeling paradigm** to construct good, reliable models. It is a very interesting example of good software engineering.

Everything you need to do proper simulations



The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

5

- Coding Part
- Structure representation
- Rapid statistics generation and analysis

Other features in this picture:

- running simulations
- debugging simulations
- random number generations (see line 143 and 157, extracting numbers according to the Weibull distribution)

Introducing the OMNeT++ modeling paradigm

Two extremely simple systems

Tin-can telephone



Point-to-point Internet communication



Let's consider two very simple scenarios composed of two nodes. Each node receives a packet from its peer, processes it and sends it back.

Tin-can telephone example: two cups connected by a wire/string.

P2P network communication: two computers connected over a generic network, e.g., the Internet

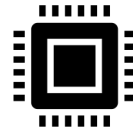
Both systems are characterized by:

- two communicating nodes: these have associated «computation time» and a «message length» parameters.
- one communication channel: this has an associated «delay» and «loss probability» parameters

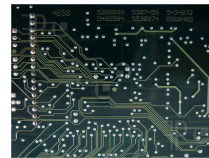
Introducing the OMNeT++ modeling paradigm

Entities involved in the model

- Communicating elements



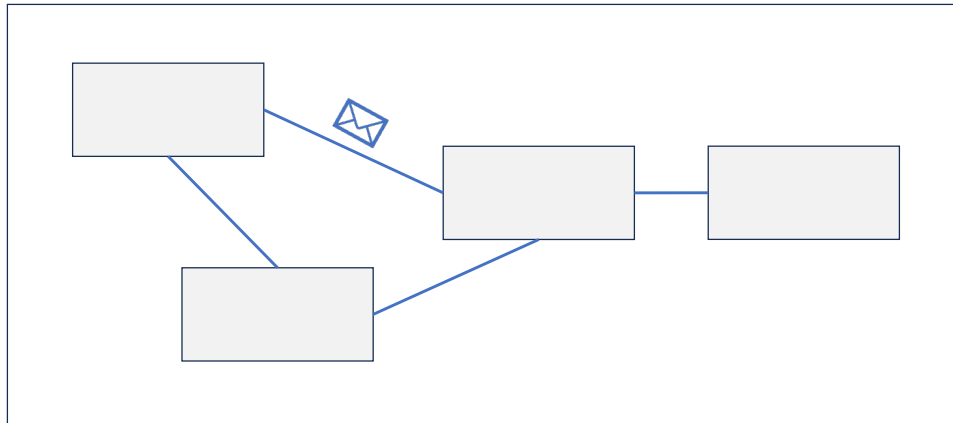
- Connections



Even more complex scenarios are generally composed of communicating elements of various kind, connected through connections (single or multiple) with very heterogeneous characteristics. Bear in mind that we are not necessarily talking about networking systems...

The OMNeT++ modeling paradigm

System (network) as a set of **modules** exchanging **messages** via **connections**



As OMNeT++ was mostly thought for networks, a system is referred to as **network**. However, you are free to model non-network-related systems.

According to the OMNeT++ philosophy, a system is composed of several **modules** exchanging **messages**, through **connections** linking their **gates**.

Modeling entities

Different things → Different files



Structure



Behavior

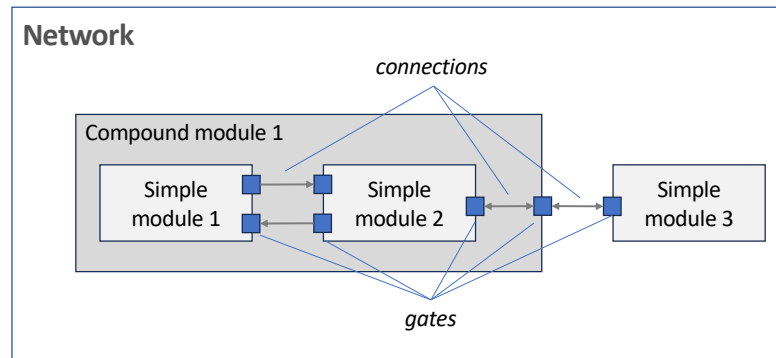


Parametrization

In order to build your model, **different things** are defined via **different files**. OMNeT++ separates a module's **structure**, **behavior**, and **actual parameter values**.

- The description (i.e., gates, connections and parameter definition) is expressed in separate files written in Network Description (NED) language. NED is a declarative language, which exploits inheritance and interfaces, and it is fully convertible into XML. NED allows one to write *parametric* simulation scenarios, e.g. rings or trees of variable size, via both a GUI (for basic/novice editing) and textual editing (for advanced/expert editing).
- The behavior is coded in object-oriented C++, and defines how the module reacts to (handles) events.
- Parameter values are written in initialization (INI) files. INI files contain the parameter values that will be used to initialize the simulation.

Network entity



- Defined in a **NED** file
- A network is the **top-level module** of the simulation

Simple modules are the basic building blocks in the OMNeT++ architecture. Simple modules can be composed together to build **compound modules**. Both simple and compound modules may have input/output interfaces, called **gates**. **Connections** can be built between gates, allowing modules to exchange **messages**.

All these elements can be arbitrarily composed together to build a **network**.

Note that in this network you do not define the behavior and the actual parameters of the modules, you just specify which entities compose your network and how they are connected between each other.

Simple module definition [NED]

```
simple Txc
{
    gates:
        input in;
        output out;
}
```



The example used in these slides comes from the TicToc tutorial available on the OMNeT++ website.

The structure of a simple module is defined via NED files. It is very similar to declarative languages (e.g. HTML) as it defines **what** something is, rather than how it behaves.

In this case we are defining a simple node called *Txc*, having one input and one output interfaces. You can define a gate of type **inout**, which can both send and receive messages.

As we will see, OMNeT follows the paradigms of Modularity and Object Oriented also from the “structure” point-of-view, like C++.

This represents the definition of a simple module of type *Txc*, just like C++ classes.

Network definition [NED]

```
network Tictoc
```

```
{
```

```
  submodules:
```

```
    tic: Txc;
```

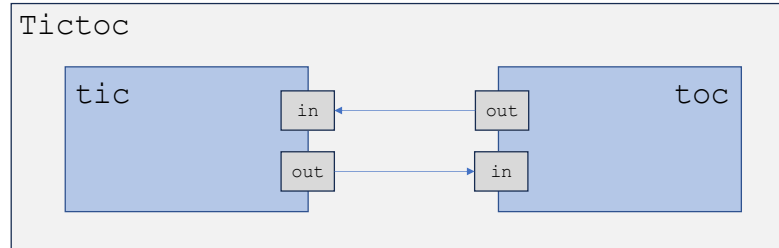
```
    toc: Txc;
```

```
  connections:
```

```
    tic.out --> { delay = 100ms; } --> toc.in;
```

```
    tic.in <-- { delay = 100ms; } <-- toc.out;
```

```
}
```



We can create two instances of the simple module Txc in order to create a network. This is done again using NED language.

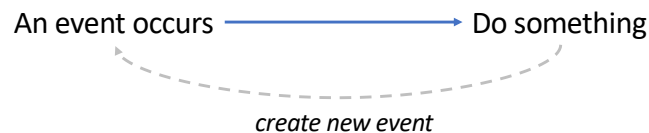
- 1) First we define the name of the network.
- 2) Then we specify the modules that will compose the network itself. Each of them has a name (tic and toc in the example) and a type (e.g., Txc)
- 3) Finally we connect the gates of the modules (use the dot notation to identify a gate within a module), specifying their connections. Connections themselves may have their characteristics (e.g. delay)

After we have defined this network, we are already done, and we can already start our simulation! However, this network does... perfectly nothing. It is just like we created two football players that are standing still in the middle of the field.

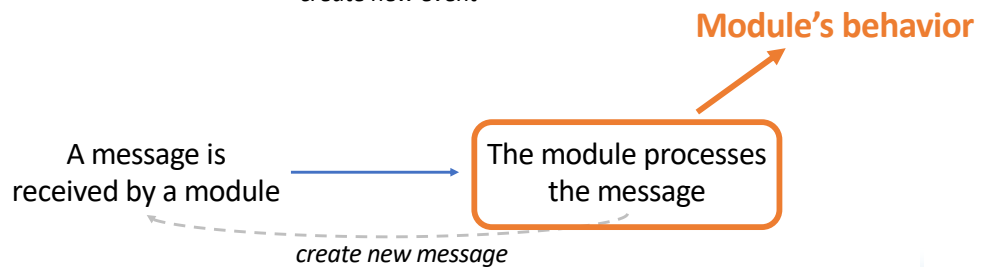
Why? Because we still need to add the **behavior** of the modules.

Modeling the system behavior

In a discrete-event simulation, the system evolves following a **cause -> effect** paradigm



In OMNeT++:



As we said, OMNeT++ is an event driven simulator. The evolution of a DES follows a cause-effect paradigm.

In OMNeT++, events are (almost always) the **reception of messages** by a module. Thus, defining the processing of a message (the effect of a message reception) means defining the behavior of a module.

The behavior of modules is defined via C++ files. Thus, each simple module must be associated with a C++ class that defines its behavior.

Compound modules do not have associated classes. Their behavior is the summation of their composing modules. As a result, also a network does not have a behavior (since a network is just the top-level compound module)

From OMNeT++'s Manual: "although the C++ class for a compound module can be overridden with the **@class** property, this is a feature that should probably never be used. When there is a temptation to add code to a compound module, then encapsulate the code into a simple module, and add it as a submodule"

Each event (or better, the handling function of that event) can create one or more new events: each event can add a new event to the timeline

Modeling the system behavior

In our `TicToc` network example:

Events	Behavior
Simulation starts	<code>tic</code> sends a message to <code>toc</code>
<code>Txc</code> receives a message	<code>Txc</code> sends a reply
(Max simulation time reached	Just terminate the simulation)

The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

14

In our `TicToc` network, we want to model a system where `Tic` and `Toc` repeatedly bounce back a message until the simulation ends, assuming that `Tic` talks first.

We identify two events to be handled (that is: two “causes”, for which we define the corresponding effect – behavior):

- simulation starts: the effects involves creating a new event if the name of the `Txc` module is “`tic`” (i.e., sending a message to the module called “`toc`”, which results in a new event to be handled by module “`toc`”)
- the `Txc` module receives a message: the effect involves creating a new event (i.e., sending back the message, which results in a new event to be handled by the other module)

Actually, there may be a third event, i.e. “max simulation time is reached”, which (in this case) makes the simulation terminate. Note that max simulation time may be not set (simulation does not end unless termination is forced, e.g., by typing `CTRL+C`)

While for the “`Txc` receives a message” event it is clear who generates it (i.e., the other module in the network), who generates the “simulation starts” event? It is automatically generated at the creation of a module (usually at the beginning of the simulation), which in turn executes the corresponding behavior. So, if you have `N` modules, you will have `N` “simulation starts” event (one for each module, and each of them executes its behavior)

Defining a module's behavior: event handlers

Each *simple* module has a **C++ class**, which should define the following functions:



`initialize()`



`handleMessage()`



`finish()`

We can have three types of events: module's creation, message reception, simulation finish (from the point of view of a module).

OMNeT++ provides us functions to handle those events (similarly to what happens with DOM events in HTML, e.g. `onLoad`, `onClick`, etc.). Such functions are called **event handlers**.

- The **initialize** function is called automatically after a module is created and is generally used to perform setup operations on a module.
- The **handleMessage** function is called whenever a message is received by a simple module. It is generally used to identify the message type, manage it, and create a new one.
- The **finish** function is called automatically at the deletion of the module from the simulation (usually, at the end of the simulation for all the modules). It is generally used to record statistics.

Defining a module's behavior: event management

Event handlers can make use of the following functions to **create new events**:



`send()`



`scheduleAt()`



`cancelEvent()`

How can we create events?

- The **send** function is used for sending out OMNeT++ messages through an output gate (i.e., to another module connected to that gate through a connection).
- The **scheduleAt** function is used by a module to send a message to itself. Why would a module send a message to itself?

In practice, the net effect of invoking *send* and *scheduleAt* functions is to add a new event with the given firing time in the event queue.

- The **cancelEvent** function is used to cancel a timer: remove an event from the event queue.

Implementing a module's behavior [C++]

```
#include <omnetpp.h>

class Txc : public cSimpleModule {

protected:

    virtual void initialize();

    virtual void handleMessage(cMessage *msg);

    virtual void finish();

};

// Module class needs to be registered with OMNeT++

Define_Module(Txc);
```

The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

17

How can we implement the cause->effect relation? That is: how does calling the send() function from a module result in executing the handleMessage() function at another module?

This is done automatically by the OMNeT environment, since each module-related class has to inherit at least from the **cSimpleModule** class.

If we use the OMNeT++ IDE commands, this is done automatically.

What is the difference between the class constructor and the initialize function? They work at two different logical levels:

- The **constructor** deals with operations related to the C++ class, as variable initializations, creation of the data structures, etc.
- The **initialize** function deals with OMNeT++ stuff and operations related to the module, as message creation and first transmission, reading parameters, etc.

This is the first example of the double nature of the module: a C++ class on one side and NED module on the other.

Define_Module pairs the C++ class to the corresponding NED module.

Since Define_Module is a macro, the compiler will generate code from that. Thus, the Define_Module should not be put into the .h file of a C++ class (use the .cpp file for that)

initialize()

Simulation starts → tic sends a message to toc



```
void Txc::initialize()
{
    if (strcmp("tic", getName()) == 0)
    {
        cMessage *msg = new cMessage("tictocMsg");

        send(msg, "out");
    }
}
```

In our example we use the initialize function to create the first event (tic sends a message to toc).

getName() is a member function inherited from the cSimpleModule class. It returns the name of the current module.

A new object of type **cMessage** is first instantiated and then it is sent through the gate *“out”*. NOTE: *out* is the name of the gate as defined in the module’s NED file.

The send() function will generate a new event at a point that depends on the connection parameters... what happens if a zero-delay connection is used?

Note that the message is sent **through the output gate**, there is absolutely no mention of the *“toc”* module who will receive the message -> this increases the flexibility of the model, because it allows you to define a new network, connecting tic to a module other than toc and you do not need to modify the behavior (the C++ code) of the module.

handleMessage()

Txc receives a message



Txc sends a reply



```
void Txc::handleMessage(cMessage *msg)
{
    // check message type
    [...]
    // send message out
    send(msg, "out");
}
```

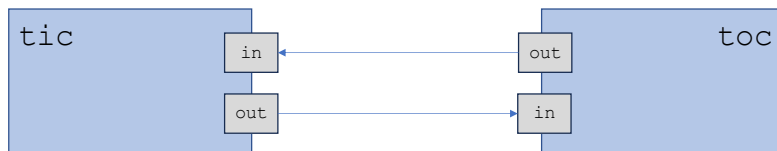
Usually, a module may receive different types of messages, so the handleMessage() should first **check what kind of message has been received**.

In this example, only one type of message exists, so there is no need to check.

In general, we could have written something like:

```
if ( msg->isName("ticTocMsg") )
{
    // do stuff
}
```

How does sending a message relate to creating an event?



What does it **actually** mean sending a message? What is the relationship of this action with the event queue?

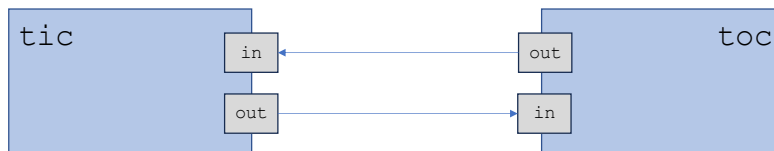
When *tic* sends a message through the “out” gate, OMNeT++ creates an event having:

- the reception time of the message at *toc* as arrival time (firing time)
- a priority
- the *toc* module as entity that will handle the message

Such event is inserted into the event queue.

When the event scheduler extracts that event from the event queue (i.e., when simulated time = arrival time of the message), it executes the `handleMessage()` function defined by the C++ class for the *toc* module

How is sending a message handled at C++ level?



How does that work in C++?

Tic and Toc are two C++ objects. Msg is a C++ object itself, as well.

Msg has a “owner” field, which indicates the module that owns it in that moment, i.e. where the message is at that time of the simulation.

Whenever a `send()` is invoked, the owner field is changed to the object implementing the event queue (as the message is inserted into the event queue); then, when the simulation time is equal to the arrival time, the owner of the message is changed to the destination module of the message. If a `scheduleAt()` was invoked, then the owner will remain the same module that added the message to the queue.

From the OMNeT++ manual: “the concept of **object ownership** is associated with the “right to delete” objects”.

The ownership mechanism is mostly used for sanity check: you will get an exception if a module tries to delete/send a message that it does not own. This will become important when we will introduce “cross-module calls”

Event scheduling

- Events (messages) are placed in an event queue called **Forward Event Set (FES)**
 - Implemented as a *binary heap*
- FES is accessed by an **event scheduler**, whose main operations are
 - `takeNextEvent()`
 - `executeEvent()`
- Both the FES and the event scheduler are configurable by extending the respective classes
 - **cFutureEventSet**
 - **cScheduler**

The default implementation of the **cFutureEventSet** uses a binary heap.

The default implementation of the **event scheduler** executes the next events in queue as soon as the execution of the previous one has completed.

The `takeNextEvent()` function “just” retrieves the next event from the FES.

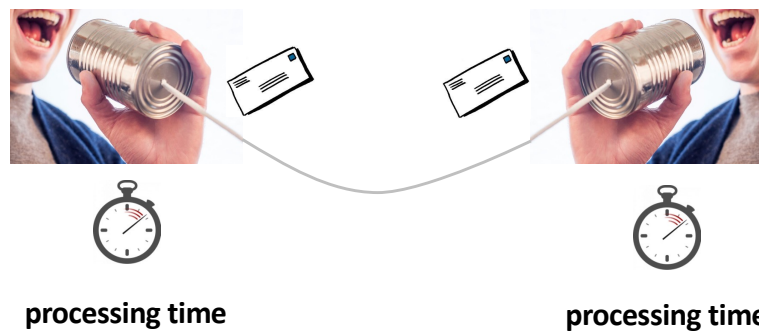
Events are extracted by arrival time (i.e., the time at which the event occurs). If two events arrives at the same time, extracts the one with highest priority. If two events have the same priority, it extracts the one inserted earlier into the FES.

Once the event has been extracted, the default behavior of the `executeEvent()` function is to call the `handleMessage()` function of the module “where the event occurs” (i.e., the destination of the message).

Several scheduler alternatives are already available in OMNeT++, including real-time and emulation schedulers.

Timers

- Sometimes, events need to be scheduled after a certain amount of time
- The module can **send a message to itself** → **timer**



We may need to model events that will occur periodically or after a variable amount of time. Those type of events can be realized via timers.

Timers are event that are thrown on a simple module after a certain amount of time.

How can we create a timer using the available events? A simple module can create a timer by **sending a message to itself**.

Adding a timer [C++]

```
class Txc : public cSimpleModule {
private:
    cMessage *beep_;
    cMessage *tictocMsg_;
    ...
};

void Txc::initialize() {
    if (strcmp("tic", getName()) == 0) {
        cMessage* msg = new cMessage("tictocMsg");
        send(msg, "out");
    }
    beep_ = new cMessage("beep");
}
```

The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

24

A possible full implementation of the code above would require each Txc module to create its own instance of beep message, e.g. into the initialize function. We recall that only the *tic* module has to create the “tictocMsg”, thus only one instance of it will be available during execution.

Adding a timer [C++]

```
void Txc::handleMessage(cMessage* msg) {
    if(msg->isName("tictocMsg") ) {
        scheduleAt( simTime() + 10, beep_ );
        tictocMsg_ = msg;           // store msg for later transmission
    }
    else if (msg->isSelfMessage()) // same as if (msg==beep_)
        send(tictocMsg_, "out");
}

void Txc::finish() {
    cancelEvent(beep_);           // or: cancelAndDelete(beep_);
    delete beep_;
}
```

The `handleMessage` function has `cMessage *msg` as actual parameter.

We have to check for the type of it as we have only one `handleMessage()` function to deal with all kind of messages.

- `isSelfMessage` is a member function of the class `cMessage` which returns true if the given message was scheduled via the `scheduleAt` function.
- `isName` is a member function of the class `cMessage` which returns true if the message name is the same as the string passed as argument
- `scheduleAt(time,msg)` sends the message `msg` to this module (i.e. the same module that is calling it). The message will be received at absolute time `time`. A variant of `scheduleAt()` is called `scheduleAfter(offset,msg)`, which instead schedule a message after a time `offset` starting from now.

Note that when the `tictocMsg` is received, a pointer to it has to be stored into the `tictocMsg_` member variable, in order to keep a reference to it and send it out in after the timer `beep` “expires”, i.e. the `handleMessage` function is called with `msg==beep`.

Defining parameters [NED]

```
simple Txc
{
    parameters:
        double procTime = default(10);

    gates:
        input in;
        output out;
}
```

A module has parameters, which can be assigned default values.

Default values will be used when running simulations in case no actual parameters are given in the network NED or INI file.

Defining parameters [NED]

```
simple Txc
{
    parameters:
        double procTime @unit(s) = default(10s);

    gates:
        input in;
        output out;
}
```

Unit of measurement

Default value

Default value can also be defined as (sub-)multiples of the unit of measurement, e.g., 0.1s = 100ms

Specifying *actual* parameters [NED]

```
network Tictoc
{
    submodules:

        tic: Txc
        {
            procTime = 1s;
        }

        toc: Txc;
    }
}
```

```
simple Txc {
    parameters:
        double procTime @unit(s) = default(10s);
}
```

In this example, *tic* gets `procTime=1s` whereas *toc* uses the default value, i.e. 10s

Using parameters [C++]

- Why using parameters?

```
void Txc::handleMessage(cMessage* msg) {  
    if (msg->isName("tictocMsg") ) {  
        scheduleAt( simTime() + 10, beep_ );  
        tictocMsg_ = msg;           // store msg for later transmission  
    }  
    else if (msg->isSelfMessage()) // same as msg==beep_  
        send(tictocMsg_, "out");  
}
```

- Avoid hardcoding parameters in the C++ code
 - Hard to modify
 - Hides default values

Always avoid writing values (of any kind) directly into the C++ code.

- 1) Hard to modify: need to recompile everytime, hence hard to use that parameter *as a factor* of the simulation study
- 2) Hidden default values: a user cannot know the value unless they read the code

Using parameters [C++]

- Read parameters from module's NED

```
void Txc::handleMessage(cMessage* msg) {  
    if (msg->isName("tictocMsg") ) {  
        simtime_t time = par("procTime");  
        scheduleAt( simTime() + time, beep_ );  
        tictocMsg_ = msg;           // store msg for later transmission  
    }  
    else if (msg->isSelfMessage()) // same as msg==beep_  
        send(tictocMsg_, "out");  
}
```

The value of a parameter can be read with the member function **par(name)**.

Representation of simulation time

- `simtime_t` type
 - Typedef of the `SimTime` class

Exponent	Resolution	Approx. Range
-18	10^{-18} s (1as)	± 9.22 s
-15	10^{-15} s (1fs)	± 153.72 minutes
-12	10^{-12} s (1ps)	± 106.75 days
-9	10^{-9} s (1ns)	± 292.27 years
-6	10^{-6} s (1us)	± 292271 years
-3	10^{-3} s (1ms)	$\pm 2.9227e8$ years
0	1 s	$\pm 2.9227e11$ years

From OMNeT Manual, section 4.1.4:

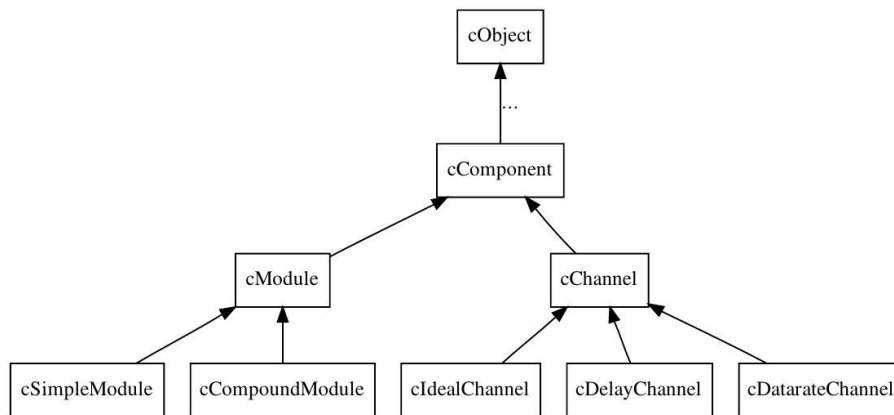
“Simulation time in OMNeT++ is represented by the C++ type `simtime_t`, which is by default a typedef to the `SimTime` class. `SimTime` class stores simulation time in a 64-bit integer, using decimal fixed-point representation. The resolution is controlled by the scale exponent global configuration variable;”

Default: picosecond. You can change it once using `setScaleExp()` method.

“Note that although simulation time cannot be negative, it is still useful to be able to **represent negative numbers**, because they often arise during the evaluation of arithmetic expressions. “

Since `simtime_t` uses 64 bit integer with configurable exponent, there is no implicit conversion to double (which uses 52 bit mantissa). You would need to use the `dbl()` method.

OMNeT++ classes hierarchy



When you include the `<omnetpp.h>` header, you can use all these classes provided the OMNeT++ simulation library.

Most objects in the simulation derives from the `cObject` (pure virtual) class, which defines several virtual member functions that are either inherited or redefined by subclasses.

Example: `getName()` function to retrieve the name of the module

The simulation library also provides useful **containers** such as **`cArray`** and **`cQueue`** classes, which can be used to store messages (e.g. to implement a queue at a network interface). They also define member functions and iterators, just like the STL.

Providing random inputs

Random number generation

Using random numbers

- By default, OMNeT++ uses the **Mersenne Twister** RNG
- Various distributions:
 - `uniform()` ;
 - `exponential()` ;
 - `normal()` ;
 -
- Importance of the **seed**

OMNeT++ gives us various random number generators.
The default one is the well known Mersenne Twister.

We do not need to worry about, say, apply the inverse transform method to generate a random variate, because OMNeT++ provides us functions that can be called to obtain a random number according to the desired distribution.

Of course we need to provide seeds to our RNG instances, but no need to worry about selecting the ones that gives us full period etc.

Using random numbers – example [C++]

```
void Txc::handleMessage(cMessage* msg) {  
    if (msg->isName("tictocMsg") ) {  
        double time = uniform(0,10);  
        scheduleAt( simTime() + time, beep_ );  
        tictocMsg_ = msg;          // store msg for later transmission  
    }  
    ...  
}
```

Assuming that Txc module has a random delay between 0 and 10, we can use a RNG to obtain a random delay uniformly distributed.

We can generate random numbers from C++ code, although the same caveats discussed earlier when hardcoding parameters apply.

Using random numbers – example [NED]

```
network Tictoc {
  submodules:
    tic: Txc {
      procTime = uniform(0s,10s);
    }
  [...]
}

void Txc::handleMessage(cMessage* msg) {
  if (msg->isName("tictocMsg") ) {
    simtime_t time = par("procTime");
    scheduleAt( simTime() + time, beep_ );
    tictocMsg_ = msg;          // store msg for later transmission
  }
  ...
}
```

We can generate random numbers also from NED code.

NOTE: in this case, the value contained in the variable `procTime` is set **only once**. Thus, consecutive calls of the function `par()` on said variable will always give the same value.

You can also use a RNG as default value in the NED file of the module, e.g.:

```
double procTime @unit("s") = default(uniform(0s,10s));
```

Text-based output

- Classic C++ approach
 - `cout` or write to output file
 - May lead to inefficient execution
- Built-in approach
 - Does not print to standard output
 - Uses **EV** as output stream
 - Several log levels

- EV_FATAL
- EV_ERROR
- EV_WARN
- EV_INFO
- EV_DETAIL
- EV_DEBUG
- EV_TRACE

```
EV_ERROR << "Connection to server is lost.\n";
EV_WARN << "Queue is full, discarding packet.\n";
EV_INFO << "Packet received , sequence number = " << seqNum << "." << endl;
EV_TRACE << "routeUnicastPacket(" << packet << ");" << endl;
```

You can write to an output file using the `fstream` library.

Text-based output can be used for recording metrics but also for debugging/verification purposes.

The destination of the EV stream depends on how simulations are run:

- in GUI mode, the log is written in the specific area of the main window.
- in command-line mode, the log is not displayed anywhere

Using log levels, it becomes straightforward to filter the output, depending on what we want to observe in a specific simulation run.

- Use EV_FATAL for messages regarding unrecoverable error (e.g., those that prevent the module to continue its operation)
- ...
- Use EV_TRACE for messages that helps you following the flow of the simulation, such as entering/leaving a specific function.

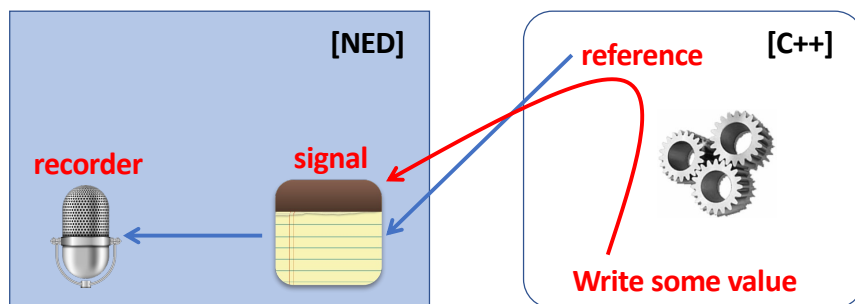
Refer to the OMNeT++ manual for complete reference.

Collecting metrics

The signal mechanism

Signal mechanism to record metrics

- Declare a **signal** in the NED file of a module
 - A signal can be *associated to a specific metric/KPI*
- When you want to “emit” a metric sample, **write** a value “into the signal”



The output of the system can be observed with **signals**. Each signal can be seen as a container for the values assumed by a certain metric during the simulation.

Signals can be of various kind, depending on the way the values are stored. At the end of the simulation, the **recorder** associated with a signal returns the metric as we defined it in the NED. These can be:

- A **scalar** value (e.g., average, sum, count)
- A **histogram** (store ranges and bins to produce a pdf)
- A **vector** (e.g., a time series representing all the values assumed by the metric over time)

Signals are defined in NED files as part of modules. They can be referred to in C++ code in order to use them for storing values.

Alternatively, you can record statistics using the simulation library directly (e.g., using/deriving the `cStatistics` class). Drawback: result recording is hardcoded in modules.

How to use signals (1/4)

- Define a **signal + statistic**

[NED]

...

parameters:

```
@signal[delay] (type=long);
```

```
@statistic[delayStat] (source=delay; record=mean;);
```

...

A statistic records data coming from a signal defined as «source».

For each statistic, you can define more than one type of recorders, e.g.: record=mean,sum,count

How to use signals (2/4)

- Define variable with type `simsignal_t`

[C++ class definition]

```
...  
  
private:  
    simsignal_t delaySignal_  
  
...
```

How to use signals (3/4)

- Register signal via **registerSignal ()**

[C++ class implementation]

```
// initialize() function
...

delaySignal_ = registerSignal("delay");

...
```

This is typically done in the initialize() function.

The string passed as argument must be the signal name as defined in the NED file. The function returns the ID of the signal, and will be used for emitting values (see next slide)

How to use signals (4/4)

- Write value into the signal via **emit(name,value)** **[C++ class implementation]**

```
// somewhere
...

emit(delaySignal_, value);

...
```

This is typically done within some event handler (i.e., handleMessage() function)

Example: recording the response time



- Send request & store t_s
 - [...]
 - Recv response & store t_r
 - Emit value $t_r - t_s$
-
- A vertical timeline diagram illustrating the measurement of response time. A vertical line has two horizontal tick marks. The top tick mark is labeled t_s and the bottom tick mark is labeled t_r . A bracket on the right side of the timeline spans the time interval between t_s and t_r , and is labeled $t_r - t_s$. Dotted arrows connect the first bullet point of the list to the t_s mark and the third bullet point to the t_r mark.

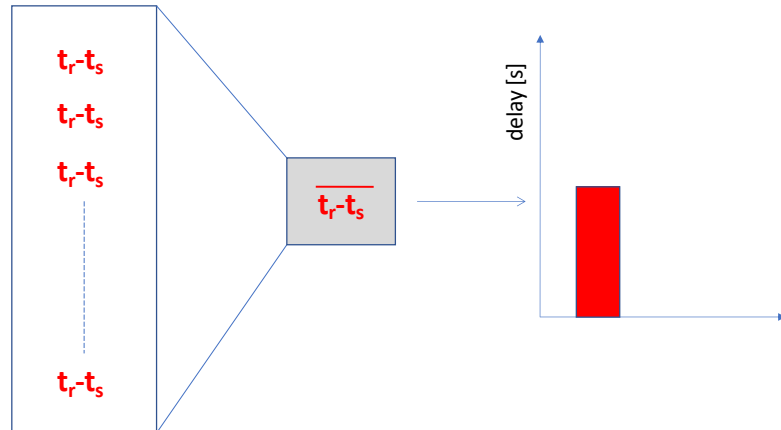
In this example, we want to measure the response time (round-trip time)

A possible way to implement this statistic in the C++ code is to let the module i) save the current simulation time to a local variable (call it t_s) when it sends a request message, and ii) get the simulation time when the response is received (call it t_r) and compute the difference with respect to the sending time (delay = $t_r - t_s$)

Example: recording the response time

Scalar metrics

```
@statistic[delayStat] (source="delay"; record=mean;);
```

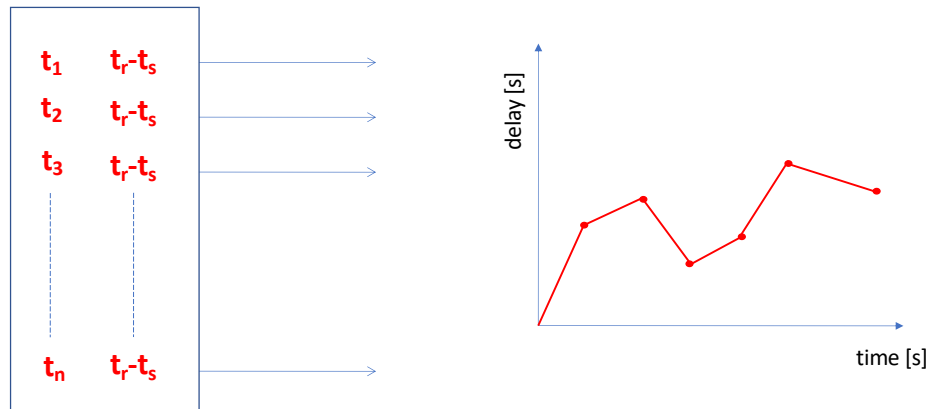


Scalar signals are a synthesis of the values assumed by a certain metric during the whole simulation. Scalar signals are useful when studying the steady state behavior of a system (e.g., an average delay) or cumulative values (e.g., the sum of sent messages). They are less useful for monitoring the behavior of a system over time.

Example: recording the response time (vector)

Vector metrics

```
@statistic[delayStat] (source="delay";record=vector);
```



It is often useful observing the “evolution” of the status of the system, i.e., the values of its metrics during its execution.

Using vectors, each value that is “emitted” using the `emit()` function, is stored in a `@statistic` together with its “emission” time.

Those values can be then plotted in a chart with their associated time on the x-axis.

Using vector can be extremely useful to spot bugs or weird behaviors.

Note: vectors eat up lots of memory resources. You may want to disable them after the debugging/verification phase.

Defining simulation scenarios

The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

a.k.a. managing (lots of) parameters and factors

Configuration files [INI]

- Allow you to assign actual parameters of a simulation
 - Overwrite (default?) values assigned in the NED file
- Managed via **INI files**
 - Organized in multiple **configurations**
 - All configurations inherit parameters from the `General` one

```
[General]
network = Tictoc
sim-time-limit = 100s
...
[Config Test1]
description = "first TicToc campaign"
Tictoc.t*c.procTime = 50ms
```

The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

All the parameters we defined for the modules can be changed via .ini configuration files. We can specify simulation-specific parameters in the same way.

.ini files are organized as follows:

- 1) A *General* configuration is first defined. The value contained here will be common to all the configurations in the file.
- 2) One or more configuration follows, each one inheriting the parameters of the general one, and possibly overwriting them

Parameters of a module are identified using the dotted notation, starting from the top-level module (i.e., the network module, `Tictoc` in this example)

Configuration files - inheritance

- Configurations can inherit parameters by **any** other configuration
- Keyword **extends**

```
[General]
network = Tictoc
sim-time-limit = 100s
...
[Config Test1]
description = "first TicToc campaign"
Tictoc.t*c.procTime = 50ms

[Config specificTest]
extends = Test1
sim-time-limit = 5s
```

Inheritance of configurations can be achieved via the **extends** keyword. This way we can organize a configuration file hierarchically.

Definining simulation scenarios

- Can be done by appropriately writing the **INI** file
- Need to define
 - Parameters
 - Factors

Once our model is ready and implemented, we have to define our simulation scenarios by specifying both simulation parameters and model factors.

This can be done with .ini files, hence without touching the network topology and/or modules' behavior.

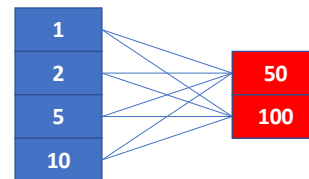
Parameters: assigning values to the module's parameters defined in NED files

Factors: the parameters whose different values we want to explore during the simulation campaign

Factors range

- Which parameters?
- Which values?

```
** .delay      = ${1, 2, 5, 10}  
** .packetSize = ${50, 100}
```



First of all we have to decide which characteristics of our system will be explored during the simulation, thus we have to decide the *factors*.

Then we have to decide the values that will be assumed by the factors.

We have two main solutions:

- 1) create an .ini configuration (i.e., one [Config] section) for each value assumed by a factor. What happens if we have more than one factor?
- 2) define a set of values for each factor

In the second case the simulation environment automatically computes the Cartesian product between all the sets of values for each parameter.

Example: running different experiments



```
*.tic.procTime = ${1, 10}s  
*.toc.procTime = ${1, 10}s
```

Run 0

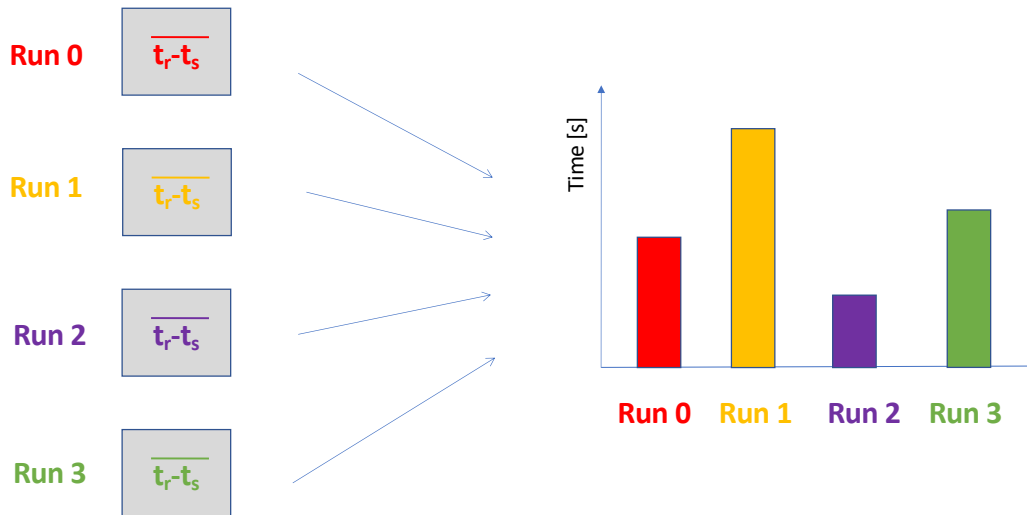
Run 1

Run 2

Run 3

In this example, we have automatically created 4 different runs, which allows us to test the network in 4 different scenarios

Example: running different experiments



The OMNeT++ Simulation Framework

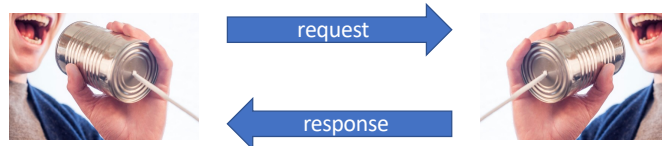
Ing. Giovanni Nardini - University of Pisa - All rights reserved

53

Each run produces its own statistics (e.g., average delay), according to what has been defined in the NED file, and you can easily compare them.

Note that running just one run for a given configuration is not enough to produce statistically-sound results in case the model uses randomly-generated inputs.

Example: running different experiments



```
repeat = 10
seed-set = ${repetition}
*.tic.procTime = ${1, 10} s
*.toc.procTime = ${1, 10} s
```

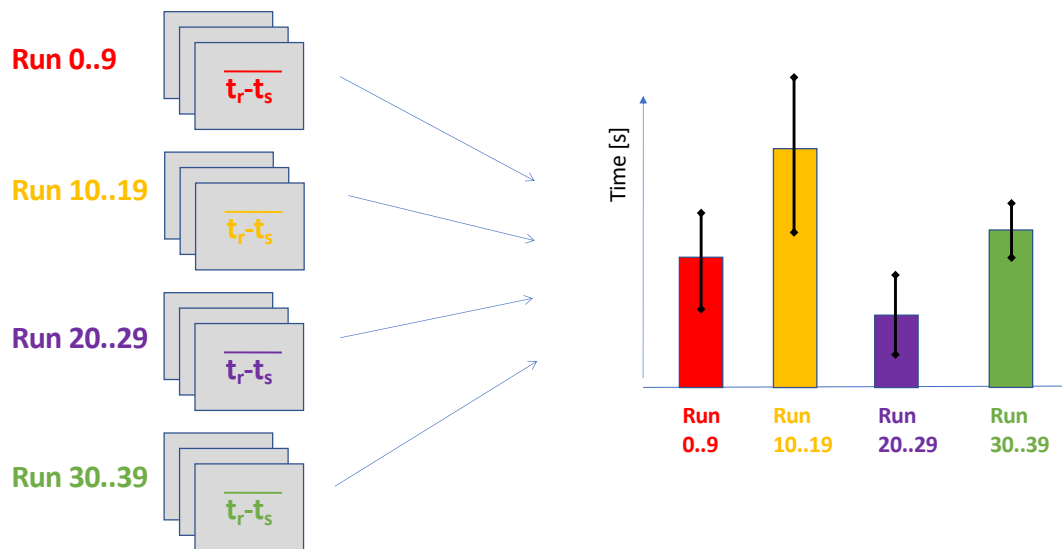
Run 0	Run 10	Run 20	Run 30
...	...	...	...
Run 9	Run 19	Run 29	Run 39

When random input is used, you need to run multiple repetitions of the same scenario, and each repetition will use different RNG seed.

In the example we specified 10 repetitions: the previous 4 configurations are repeated 10 times each, and each repetition initializes the RNG with a different seed, in order to obtain independent samples.

By default, each run sets the seed equal to its run ID (0,1,2,...,39). However, for fairer comparison, it is useful to make each i-th repetition of each scenario to use the same RNG seed. This is done by setting the seed-set parameter to `${repetition}`

Example: running different experiments



The OMNeT++ Simulation Framework

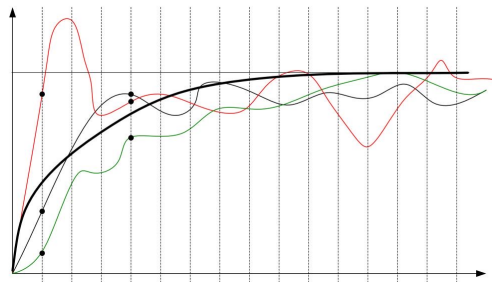
Ing. Giovanni Nardini - University of Pisa - All rights reserved

55

Now, for each configuration we can average the results produced by the 10 independent repetitions and plot the confidence interval as well.

General simulation parameters

- Duration: `sim-time-limit`
- Warm Up: `warmup-period`
- Repetitions: `repeat`
- Define Seed: `seed-set`



These parameters are always available, regardless the network we want to simulate

Handling output files

The output files (.sca and .vec) will be stored into the folders defined by the **output-scalar-file** and **output-vector-file** parameters that, by default, are set as:

```
output-scalar-file = ${resultdir}/${configname}-${runnumber}.sca
```

```
output-vector-file = ${resultdir}/${configname}-${runnumber}.vec
```

In order to make it simpler to link a specific output file to a simulation configuration (remember the problem we discussed in the simulation workflow), we can assign names to **iteration variables**, e.g.:

```
**tic.delay = ${dtic=1,10}
```

```
**toc.delay = ${dtoc=1,10}
```

By changing the output file parameters as follows, we can automatically **include the iteration variables in the name of the output files**:

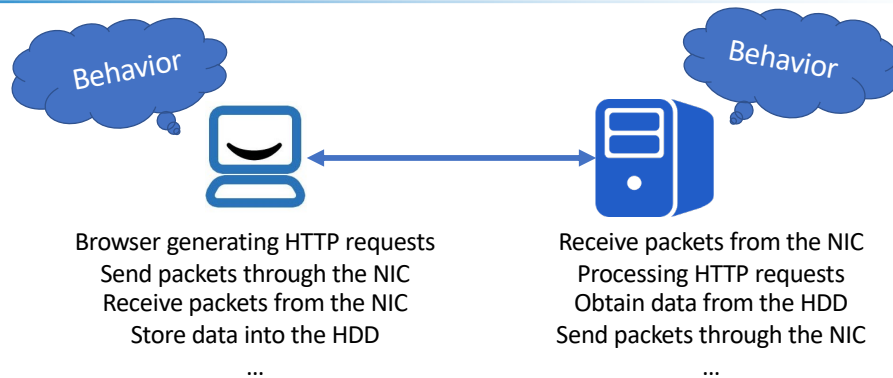
```
output-scalar-file = ${resultdir}/${configname}/${iterationvars}-${repetition}.sca
```

```
output-vector-file = ${resultdir}/${configname}/${iterationvars}-${repetition}.vec
```

Note that if we forget to add `${repetition}`, all the repetitions for the same combination of iteration vars will collide to the same output file

Compound modules

Modeling (slightly) more complex systems

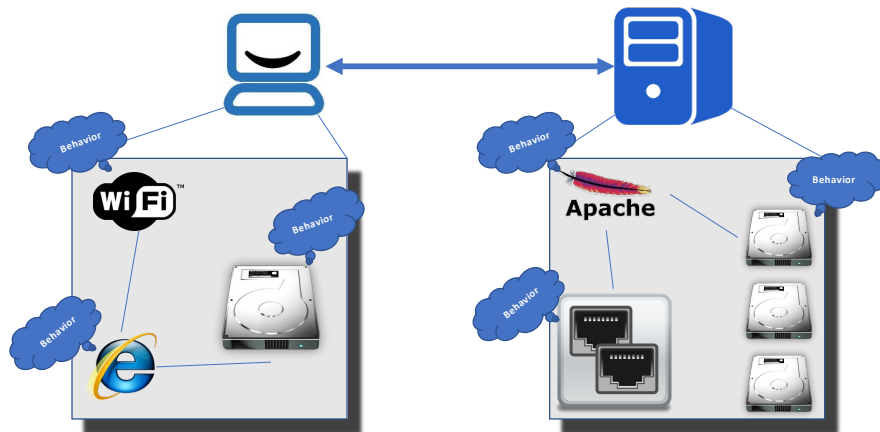


- The above client-server system can be simulated by modeling nodes as *simple modules*
- Behavior is "atomic"

If we consider a client-server architecture from a high level perspective, we just need to consider the behavior of the involved nodes as a whole.
We can model them as simple modules, thus we can associate to them a behavior that is atomic.
However, this is only a matter of modeling...

Modeling (slightly) more complex systems

- Let's open the box...



The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

60

If we want to take a deeper look into each node, we have to model the components within each of them (i.e. HardDisk, network interfaces, applications). Each component will be characterized by its own behavior thus it can be modeled as a simple module.

This way the original nodes (Client and Server) become compound nodes, with a behavior that is no more atomic: it is the result of the composition of the behaviors of their own components.

For example, we can still have the "generate HTTP request" behavior: it will start in the browser than propagated through the wifi etc.

Note that once we develop the model of a module, we can reuse it as a component of many compound modules (e.g., the same Hard Disk in this model can be used in both the client and the server)

Compound modules

```
module Pc  
{
```

```
  parameters:  
    int cpuSpeed;  
    int monitorSize;
```

```
  submodules:
```

```
    hd : HardDisk;  
    wifi: Wifi;  
    explorer: Browser;
```

```
  gates:
```

```
    ...
```

```
  connections:
```

```
    ...  
}
```

```
simple HardDisk  
{  
  ...  
}
```

```
simple Wifi  
{  
  ...  
}
```

```
simple Browser  
{  
  ...  
}
```

Some characteristics of a compound model may be out of the scope of our study/evaluation; thus, we can model them as just NED parameters (e.g., the monitor size).
Connections section will define how the submodules are connected with each other, and how they are connected to the compound module gates (for communicating with external modules)

Navigating the modules hierarchy

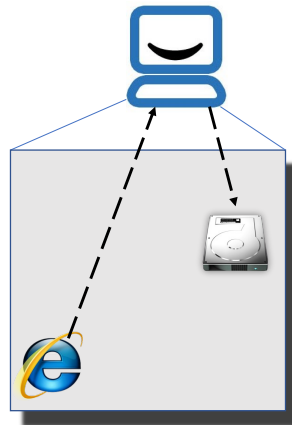
- In the C++ code, it is possible to obtain references to other modules

```
// within Explorer C++ class
cModule * pc;
pc = getParentModule();

int size = pc->par("monitorSize");
...

cModule * hd = pc->getSubmodule("hd");
int hdSize = hd->par("hdSize");

// [cast hd to HardDisk*]
int size = hd->getSize();
```



As we saw, compound modules have their own NED parameters. A simple module within a compound one may want to obtain these values: e.g. a browser that wants to know the screen size/resolution of the PC. A module that is contained into another one can obtain a pointer to it (namely to its parent) with the function `getParentModule`. It can then use it to call all the public functions of the parent (e.g. calling `par("name")`).

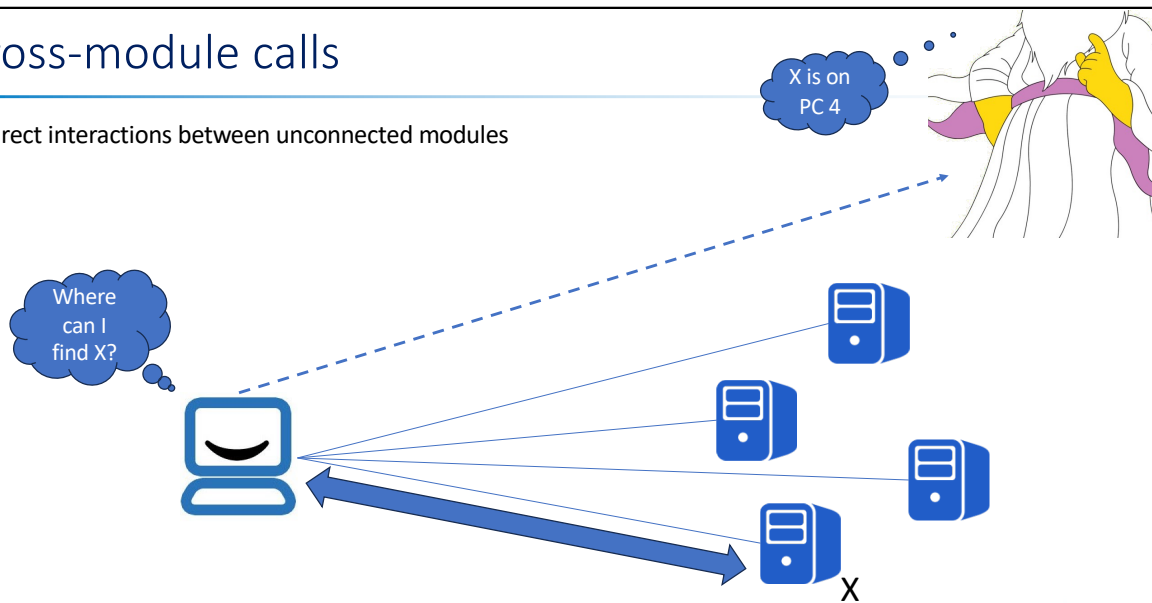
You can also obtain the reference to a submodule by specifying its relative or absolute path, i.e. the sequence of submodules to traverse to reach that submodule (from the top-level module in case of the absolute path, from the submodule calling the function in case of relative path).
E.g.: `findModuleByPath("pc2.hd1")` // or `getModuleByPath` in OMNeT++ versions < 6.0

Use with caution! You need to know the actual name of the submodule in your network scenario ("hd1" instead of "HardDisk"), hence the behavior of your submodule might be wrong in case you assign the submodule a different name.

As a "best practice" it is better to wrap parameter requests from other modules into getter/setter function, e.g. to add error control

Cross-module calls

- Direct interactions between unconnected modules



Sometimes there may be the need for two nodes to interact without messages (e.g., unconnected modules). A module can keep a reference to another module, in particular it can keep a pointer to the cModule class of that node.

Let's suppose that a PC wants to obtain resource X from a server, but it does not know where X is actually located. We may model this with a *discovery* procedure between the PC and the server, but it might not be the goal of our study.

Thus, we can assume to have a "supernatural entity" of the system who knows everything (it is quite common to hear about the "oracle").

As far as we do not need to model the communication between the PC and the oracle, we may mimic this procedure with a function call... but this is something that has to be used carefully as it may lead to unexpected behaviors.

Cross-module calls [C++]

```
// somewhere in C++
cModule * temp;
temp = findModuleByPath("oracle");

OracleClass * oracle;
oracle = check_and_cast< OracleClass* >(temp);

int serverId;
serverId = oracle->aSearchFunction("x");
```

We can obtain a reference to any cModule within our network by calling the findModuleByPath("name") function.

We have then to cast the resulting cModule * into the proper pointer with check_and_cast<type *>. The check_and_cast function performs a dynamic_cast and checks for errors.

Cross-module calls

USE WITH CAUTIONS!!!



Use only getter/setter

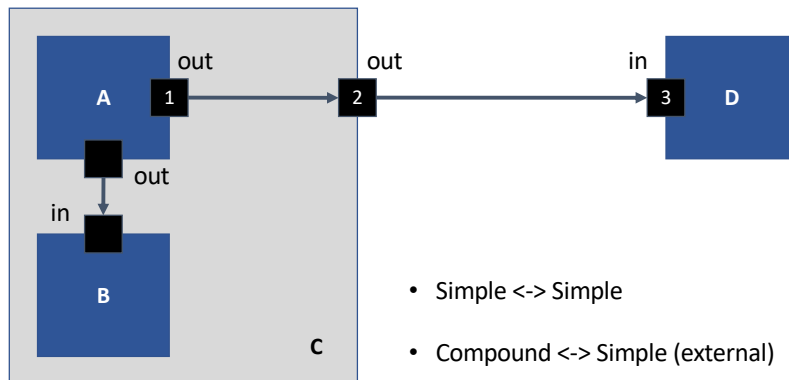
```
getSomething( name );
```

```
setSomething( name , value );
```

Bear in mind that we are modeling a system in terms of events, thus all our elements are built to work in that way.

Normally a module modifies its own data structure when an event occurs. Also, the ownership mechanism works as long as messages are exchanged using the `send()` function. Cross calls work asynchronously and may let modules to mess with other modules data structures, or send messages from a module that does not own it. For this reason, it is preferable to use cross module calls in a very controlled way, e.g., using `get/set` functions only.

Compound modules – internal connections



- Simple <-> Simple
- Compound <-> Simple (external)
- Compound <-> Simple (internal)

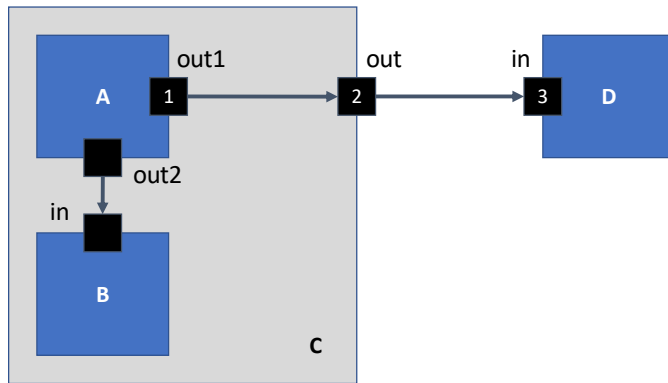
When considering simple modules, we always connected gates according to the pattern *output* → *input*. This idea allows us to send something *through* gates rather than *towards* nodes. This way we may add new modules/connections in the path between source and destination. (e.g., browser → OS → driver → wifi → dest node). When considering compound modules this is still true for *external connections*, i.e., connections with other modules (both simple or compound) **outside** the module being considered, but it is slightly different for connections within a compound module.

As the compound module C has no *autonomous* behavior associated with it, it needs a way for “generating packets”.

The above example shows two separated modeling decisions:

- connect node C and D via gates 2 and 3 (thus using gate 2 as output gate), regardless of their internal composition
- forward every packet that comes out of gate 1, through gate 2, regardless of their final destination.

Compound modules – internal connections



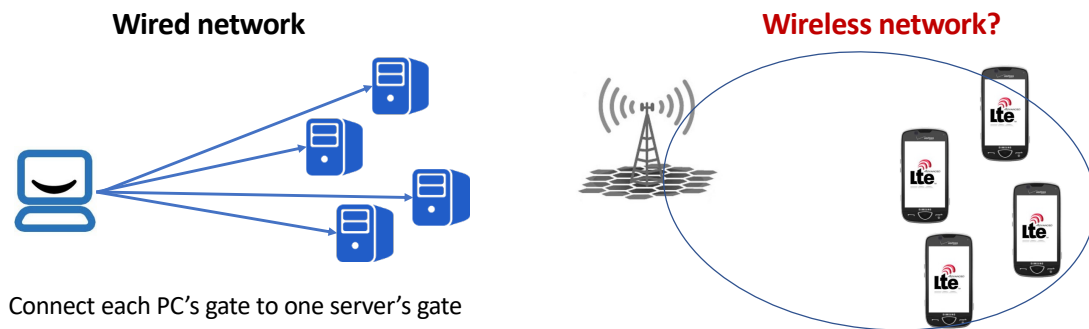
```
module C
{
  parameters:
    ...

  gates:
    output out;

  submodules:
    a: A;
    b: B;

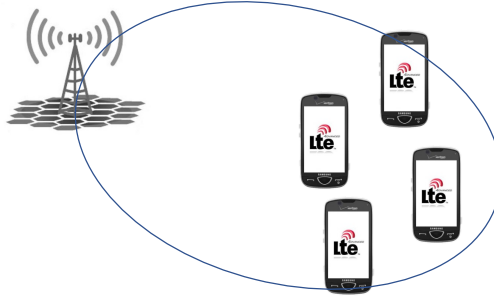
  connections:
    out <-- a.out1;
    a.out2 --> b.in;
}
```

Connecting gates



If you want to model wireless communications, it is infeasible to use connections linking gates (or, at least, it is non-natural to think about wireless connections that way)

Direct transmissions



```
[... somewhere in NED ...]  
input wirelessGate @directIn;
```

```
[... somewhere in C++ ...]  
destModule = <...>  
destGate = <...>  
sendDirect(msg , propDelay , txDuration , destModule , destGate)
```

Modules intended to receive messages via a wireless connection have an input gate specifying the property **@directIn**.

Such property specifies that a module can receive messages on that gate even if there are no connections attached to it. Basically the paradigm slightly changes: instead of sending a message towards an output gate (and from there, connections will deliver the message to the intended destination), the sending module sends the message **directly** to the input gate of the receiving module.

The pointer to the destination module can be found, for instance, using the functions for navigating the modules hierarchy like *findModuleByPath()*. Alternatively, you can take advantage of an oracle module (accessed via direct method calls) and retrieve the pointer from there.

Code modularity

```
void Txc::handleMessage(cMessage *msg)
{
    if (msg->isName("tictocMsg"))
        A very complex code ...

    if (msg->isName("beep"))
        Even more complex code ...
}
```

We are getting to the point in our career where our code tends to be pretty big: we need to design and write it in such a way that it does not *explode*.

The `handleMessage()` function is generally the core of a module behavior. It is called for **every** packet that reaches the node; thus it has to firstly check the type of the given incoming type.

However, it can rapidly grow to hundred of lines if not properly designed. This of course leads to a code that is very hard/impossible to modify/debug/understand.

Structuring the code

```
void Txc::handleMessage(cMessage *msg)
{
    if(msg->isName("tictocMsg"))
        handleTictocMsg(msg);
    if(msg->isName("beep"))
        handleTimer(msg);
}

void Txc::handleTictocMsg(msg) { ... };
void Txc::handleTimer(msg) { ... };
```

Structuring code modularly is a best practice... thus we need to structure the handle message function modularly. We should:

- check for all the expected message types,
- define a separate function for managing each type of message

Structuring code modularly simplifies a lot reuse of code. It also makes it simpler to redefine the actions to be taken upon reception of one message if we extend the class to specialize its behavior (i.e., using polymorphism): indeed, it is easier to redefine the behavior of a subset of small functions (the ones you really need to redefine) instead of redefine a one large handleMessage() function, which would likely be the same as the base one except for a few lines of code.

Handling unexpected conditions

```
void Txc::handleMessage(cMessage *msg)
{
    if(msg->isName("tictocMsg"))
        handleTictocmsg(msg);
    if(msg->isName("beep"))
        handleTimer(msg);
    else
        throw cRuntimeError("Unrecognized message type. Abort");
}
```

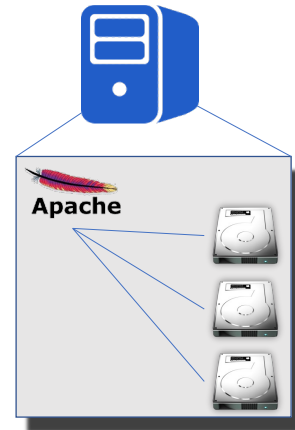
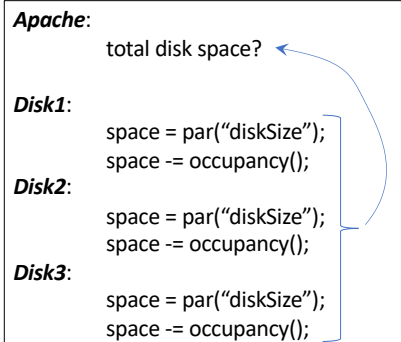
Verify the correctness of your model.

Throw exceptions whenever an unexpected condition occurs (e.g., a state in which the system should not be): this allows you to discover anomalies in your model/code.

The OMNeT++ environment has an option that allows you to automatically put a breakpoint when an exception occurs, so that you can debug the program and see the state of variables just before the exception.

Multi-stage initialization

During each initialize...



Example. A server is composed of a web server application *A* and 3 hard disks *D1*, *D2* and *D3*. *A* needs to know at the beginning the total available space within the server. Each disk has a NED parameter that specifies its available disk space. Hard disks can read this information during initialize, subtract the current disk occupancy, and store it into a variable called *space*. *A* may request the value of this variable via a cross module call during its initialization.

What happens if the initialize function of *A* is executed **BEFORE** the initialize function of *D1*, *D2* and *D3*?

Another exemplary use case where multi-stage initialization comes in handy is networking protocol stacks. For example, the IP routing protocol can compute the routes and populate forwarding tables only after all the network interfaces of the host initialized themselves and obtained an IP address.

Multi-stage initialization

- OMNeT++ may invoke each initialization function multiple times, aka **stages**

```
// in class declaration (.h)
virtual void initialize(int stage);
virtual int numInitStages() const { return 2; }

// in initialize definition (.cpp)
void MyModule::initialize(int stage)
{
    if( stage == 0 )
    { ... }
    else if( stage == 1 )
    { ... }
}
```

We can ask the system to perform initialization in multiple stages, i.e., calling the initialize function multiple times.

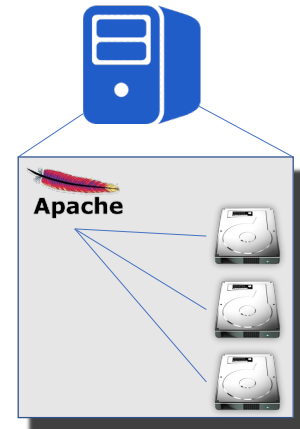
We can redefine the function **numInitStages()** in order to specify the number of initialize calls for that module (that is, each module can define its own number of stages). The **initialize()** function will then be called every time with a different (increasing) value of the stage argument.

NOTE: in order to make this work, we have to redefine **exactly** the two functions above.

Multi-stage initialization

```
// in initialize definition (.cpp)
void HardDisk::initialize(int stage)
{
    if( stage == 0 )
        space = par("diskSize") - getOccupancy();
}

// in initialize definition (.cpp)
void Apache::initialize(int stage)
{
    if( stage == 1 ) {
        ...
        totalDiskSpace = hd1->getSpace() +
                        hd2->getSpace() +
                        hd3->getSpace();
    }
}
```



NED inheritance

```
simple Txc
{
  parameters:
    bool sendMsgOnInit = default(false);
  gates:
    input in;
    output out;
}

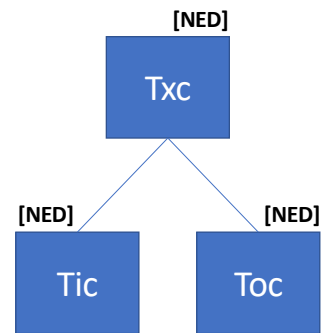
// in the .ini file
*.tic.sendMsgOnInit = true
*.toc.sendMsgOnInit = false
```

in the TicToc example, we could specify which node is going to talk first by setting the value of the NED parameter `sendMsgOnInit`, instead of checking the module's name.

This procedure is error-prone: it is not difficult to set wrong parameters or forget one in the INI file.

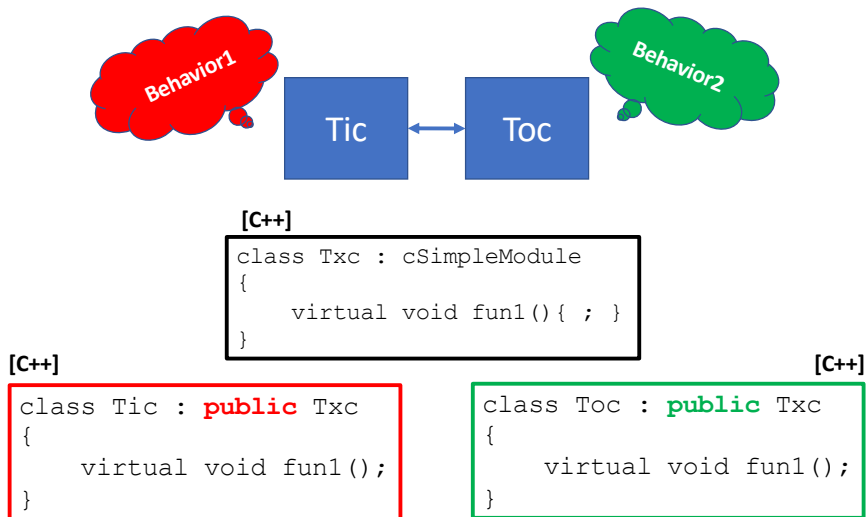
NED inheritance

```
simple Txc {  
  parameters:  
    bool sendMsgOnInit = default(false);  
  gates:  
    input in;  
    output out;  
}  
simple Tic extends Txc {  
  parameters:  
    sendMsgOnInit = true;  
}  
simple Toc extends Txc {  
  parameters:  
    sendMsgOnInit = false;  
}
```



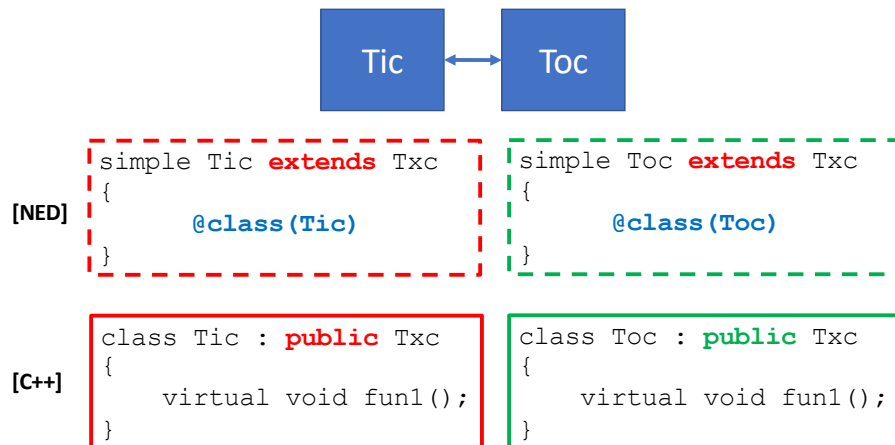
We can create two inherited modules with the same structure as Txc (gates, parameters, etc.), but with two different values for the parameter `sendMsgOnInit`. In this case a module of type Tic will always start first, whereas a module of type Toc will always wait for a message from some other module.

NED inheritance



We can apply the same reasoning to the behavior of a simple module, that is its C++ class. Thus, we can use class inheritance in exactly the same way we would do with normal C++ development. Note that if you structured your code with several small functions it is now easy to redefine the behavior of the subset of functions you need to redefine.

NED inheritance



By default, the derived NED module type will inherit the C++ class **from its base**, so it is important to remember that you need to write out **@class** if you want it to use the new class.

This is not automatic because this allows one to just extend to NED file without being forced to redefine also its C++ class even if it is not necessary. If you think about it, the Tic and Toc module would not need to redefine its behavior, since their NED definition just assign the value to a parameter of the base Txc module.

Messages definition

Customizing the data exchange between modules

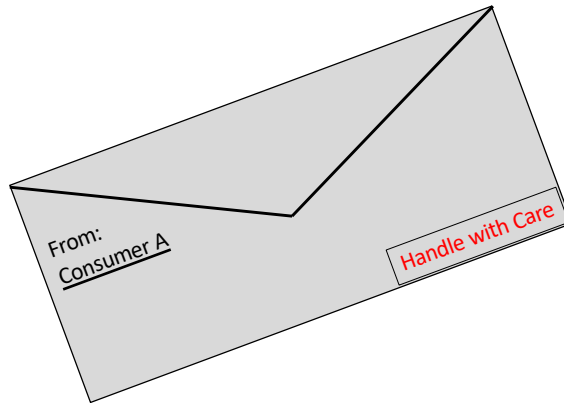
The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

So far, we have seen messages «just» as a way to trigger an event at a specific module. OMNeT++ messages **can also include data** that needs to travel from the source module to the destination module. This is actually what one would expect from a message, that is carrying data.

Messages

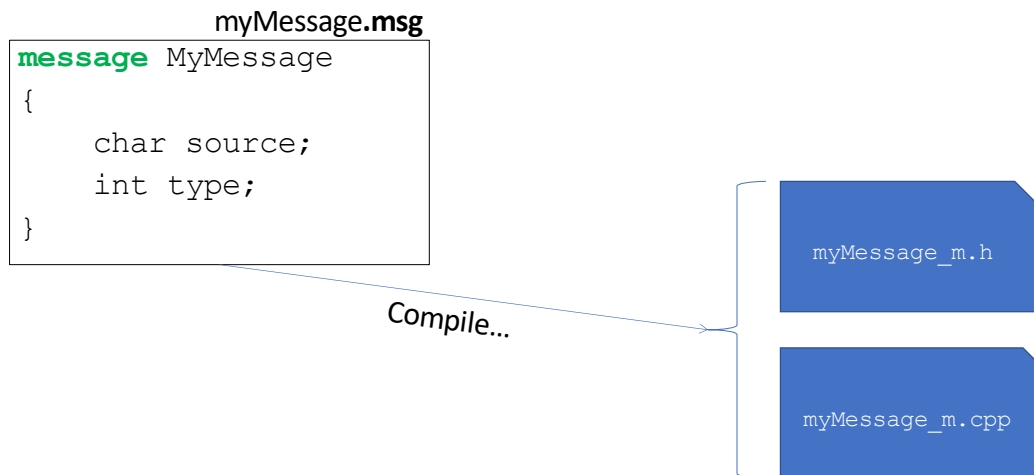
- OMNeT++ provides a `cMessage` class, which can be used to redefine our own messages
- No need to create a new C++ class manually



Since each OMNeT++ message is an instance of the `cMessage` class, one could just create a new C++ class that extends the `cMessage` one, as in standard C++ programming, and make it include new fields along with the corresponding setters and getters functions.

E.g., we may want to add the sender of a message, its priority, the type of message, etc., in order to forward it (and handle it at the destination) according to some kind of policy.

Creating a new type of message



We can define new messages by creating a **message file** (.msg) specifying the message name and the type/name of its fields. The syntax of a .msg file is the same as NED. When we build our project, the OMNeT++ environment will **automatically create a new class** (with .h and .cpp files), derived from `cMessage` and extended with the new fields (note the `_m` suffix). The fields will be added to the new class, along with the corresponding `get/set` functions

Using messages

```
#include "myMessage_m.h"
```

```
//somewhere into module A  
MyMessage *msg = new MyMessage(msgname);  
msg->setSource('A');
```

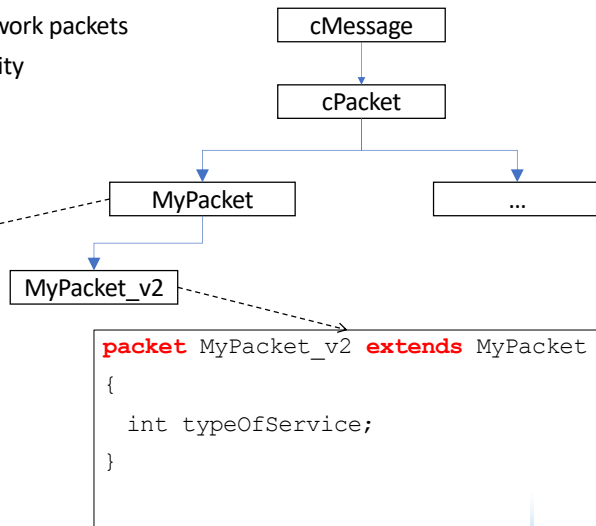
```
//in handleMessage() of module B  
MyMessage *tempMsg;  
tempMsg = check_and_cast<MyMessage>(msg);  
char source = tempMsg->getSource();  
...  
delete tempMsg;
```

In the handleMessage(), remember to cast the message to the correct type before trying to access its fields (the handleMessage function always has a cMessage* type as an argument)

Messages vs packets

- Extend the `cPacket` class if you want to model network packets
- Adds length, bit error flag, and encapsulation capability

```
packet MyPacket
{
    int timeToLive;
    int srcAddress;
    int destAddress;
}
```



`cPacket` extends the `cMessage` class by adding methods that are typically useful in network simulations, such as:

- `get/setBitLength`
- `get/setBitError`
- `encapsulate/decapsulate`

A packet is defined within a `.msg` file using the **packet** keyword (instead of **message**).

Like messages, the compiler will create `MyPacket_m.h` and `MyPacket_m.cc` file, where the `MyPacket` class extends the `cPacket` class (instead of `cMessage`)

Using multiple RNG instances

RNG seeds and repetitions

- Automatic seed selection

```
**node.size = ${s=20,200} [INI]
**node.lambda = ${lambda=10,100}

repeat = 2

seed-set = ${repetition}
```

By default, seed-set = \${runnumber} → each run gets a different set of seeds

With seed-set = \${repetition}, repetitions 0 of each experiment gets the same set of seeds

Independent random number streams

- It is possible to configure and use a number of RNG instances, i.e. independent random number streams
- OMNeT++ defines a configurable number of **physical RNGs**
 - One, by default
- Each module has its own set of **local RNGs**
- A module-local RNG must be **mapped** to one physical RNG
 - By default, every module-local RNG is mapped to the same physical RNG

OMNeT++ can be configured to make several RNGs available for the simulation model. However, usually model code does not directly work with those **physical** RNGs. Instead, there is an indirection step introduced for additional flexibility. When random numbers are drawn in a model, the code usually refers to **component-local** or logical RNG numbers. These local RNG numbers are mapped to global RNG indices to arrive at actual RNG instances.

RNG: case 1

- Node generating random-sized packets, at random time intervals

```
size = uniform(s0, s1);  
nextPacketTime = exponential(1/lambda);
```



One local RNG mapped to one physical RNG

In this case, two random variables extracts number from the same stream (!)

RNG: case 2

- Node generating random-sized packets, at random time intervals

```
num-rngs = 2 [INI]
**.node.rng-0 = 0
**.node.rng-1 = 1
```

```
size = uniform(s0, s1, 0);
nextPacketTime = exponential(1/lambda, 1);
```



Two local RNGs mapped to two physical RNG

In this case, two random variables extracts number from a dedicated stream

If not specified, local RNG 0 is used.

RNG: case 3

- Two nodes generating random-sized packets, at random time intervals

```
num-rngs = 2 [INI]
**nodeA.rng-0 = 0
**nodeA.rng-1 = 1
**nodeB.rng-0 = 0
**nodeB.rng-1 = 1
**node*.rng-0 = 0
**node*.rng-1 = 1
```



```
size = uniform(s0, s1, 0);
nextPacketTime = exponential(1/lambda, 1);
```

In this slide: four local RNGs mapped to two physical RNGs

RNG: case 4

- Two nodes generating random-sized packets, at random time intervals

```
[INI]
num-rngs = 4
**.nodeA.rng-0 = 0
**.nodeA.rng-1 = 1
**.nodeB.rng-0 = 2
**.nodeB.rng-1 = 3
```



```
size = uniform(s0, s1, 0);
nextPacketTime = exponential(1/lambda, 1);
```

In this slide: four local RNGs mapped to four physical RNGs.

Hint: using wildcards and expressions in INI file lets you save time and lines of code:

```
...
num-rngs = ${N} * 3
*.node[*].rng-0 = ancestorIndex(0) * 3 + 0
*.node[*].rng-1 = ancestorIndex(0) * 3 + 1
*.node[*].rng-2 = ancestorIndex(0) * 3 + 2
...
```

Underlying message: decoupling local and physical RNGs makes the model implementation totally independent from the number of physical RNGs.

You do not need to *always* use multiple RNGs, but you should do that when your simulation involves **independent stochastic components** whose random draws should not affect each other's sequence or when reproducibility across configuration changes matters.

For example, if you are measuring the **overall system throughput**, it is not necessary for each node to have an independent RNG instance for traffic generation, because you are only interested in the **aggregate statistic**. In that case, the exact order of packet generation among nodes does not affect the global throughput.

Conversely, if you care about **per-user throughput** or **fairness**, different random sequences among users can alter individual results or introduce biases. In such cases, assigning a **per-node RNG** is recommended to ensure independence and reproducibility.

Another case where multiple RNGs are important is when **comparing different algorithms or policies** (e.g., schedulers). Different algorithms may change the order in which random events occur, leading to different random samples even under the same configuration. Using independent RNGs ensures that any difference in the results reflects only the algorithmic changes, not random fluctuations in traffic or channel behavior.

Debugging

The OMNeT++ Simulation Framework

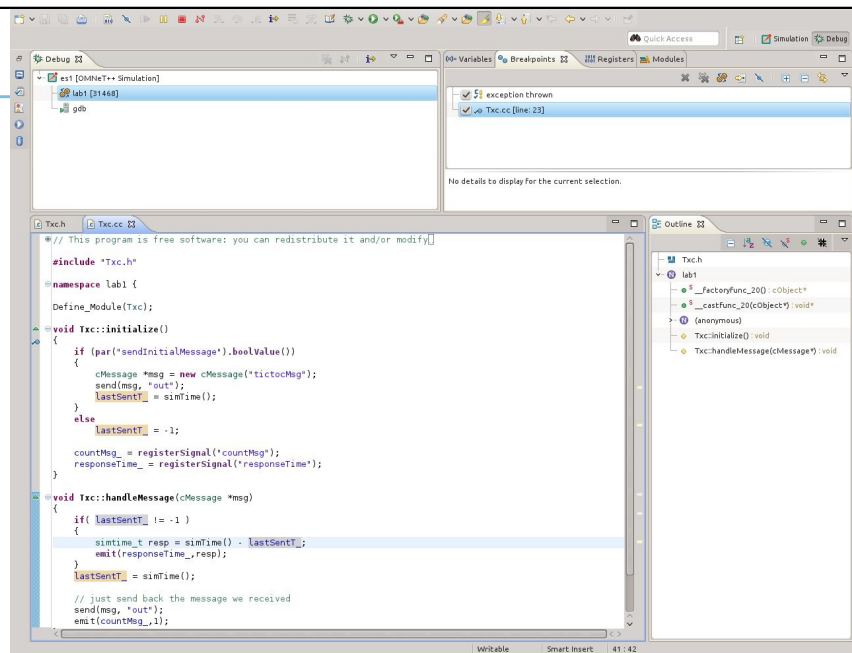
Ing. Giovanni Nardini - University of Pisa - All rights reserved

Debugging

- Execution
 - Bad **memory accesses**
 - ∞ loops
- Behavioral
 - Is your code **behaving** as the model in your mind?
- Performance
 - Memory **leaks**
 - **Profiling**

Execution debug

- Can run and control the debugging tools directly from the IDE



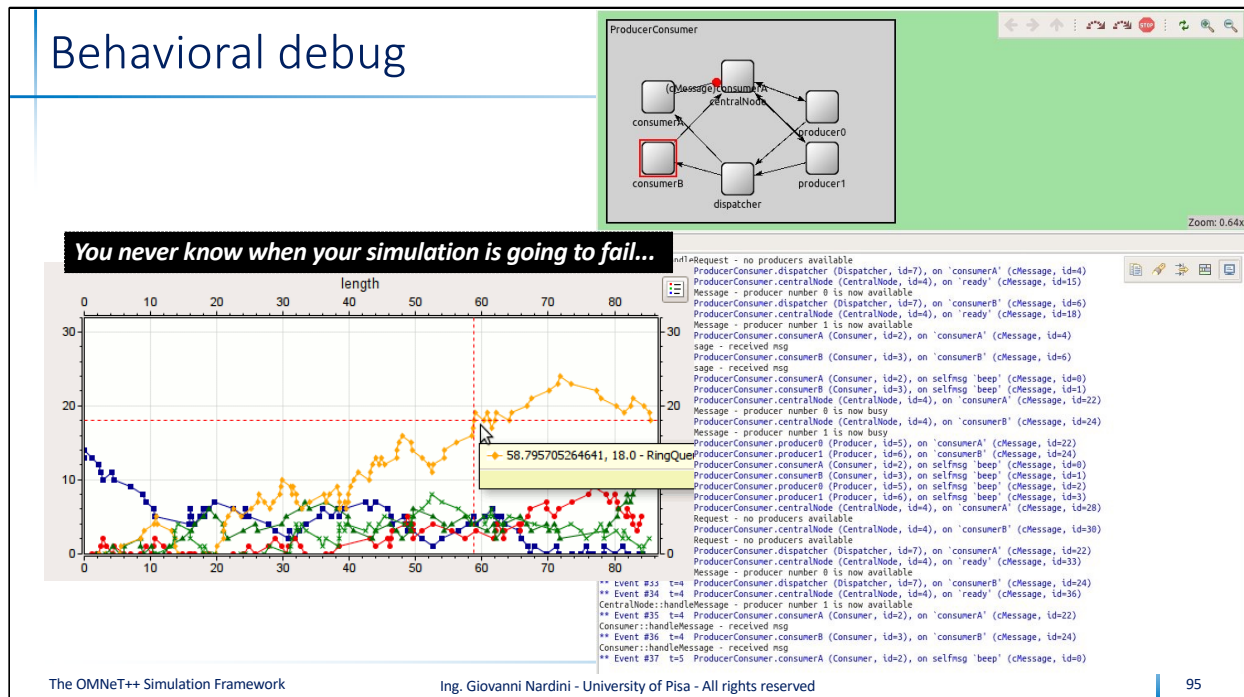
The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

94

The OMNeT++ IDE is based on Eclipse, thus it has the same capabilities in terms of debugging tools. We can set a breakpoint in the code by double clicking on the left side of a line of code (see the small blue circle). Then, we can start performing a simulation in debug mode by using the “bug” button in the menu toolbar in the same way as we do with “green and white play button” We can also add conditional breakpoints on certain cases: e.g., when a variable has a certain value

Behavioral debug



Behavioral debugging can be performed

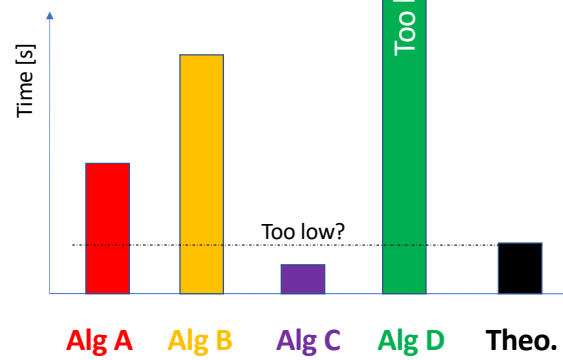
- using the visual environment (QtEnv) and observing the actual flow of packets
- printing out log information with the EV command then checking the log
- using vectors in order to find “strange” behaviors (e.g., at 40s the yellow line starts to grow strangely...)

Possible approach:

- Run a simulation and plot scalar metrics → some value looks weird
- Plot vector metrics → find when the «strange» behavior occur
- Look at the log at the time of the «strange» behavior

Behavioral debug

- Use formal/visual validation



Use scalar statistics for checking the steady state behavior of our system.
Use (if available) well known theoretical-optimum values as comparison.

Performance debug

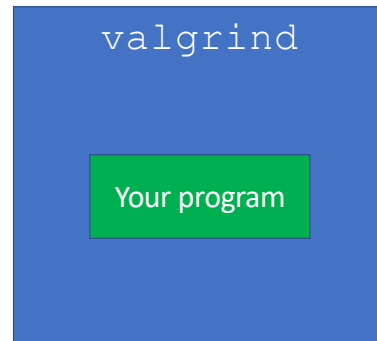
- Checking performance
 - Total execution time
 - Time spent in each function
 - Calls per function
 - Memory problems
- Tools
 - **Valgrind** → Memory issues, e.g., segmentation faults, leaks
 - Valgrind + **Callgrind** → Profiling
 - Valgrind + **Massif** → Memory heap usage
 - Alternatively, **Heaptrack** (available on Github)

Consider performance in your project as well

Valgrind

- Memchecker
- Used to
 - Detect memory management bugs
 - Does “segmentation fault” ring a bell?
 - Profile your program
 - Heap memory allocations

```
$ valgrind ./executable
```



Valgrind is a tool for checking memory issues and profiling the execution of generic software. Your simulation model is a software, so you can use Valgrind on it.

Valgrind – memory leak

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int * array1 = new int[1000];
7
8     cout << "hello" << endl;
9     return 0;
10 }
```

```
==32307== Command: ./testValgrind
==32307==
hello
==32307==
==32307== HEAP SUMMARY:
==32307==    in use at exit: 4,000 bytes in 1 blocks
==32307==    total heap usage: 1 allocs, 0 frees, 4,000 bytes allocated
==32307==
==32307== LEAK SUMMARY:
==32307==    definitely lost: 4,000 bytes in 1 blocks
==32307==    indirectly lost: 0 bytes in 0 blocks
==32307==    possibly lost: 0 bytes in 0 blocks
==32307==    still reachable: 0 bytes in 0 blocks
==32307==    suppressed: 0 bytes in 0 blocks
```

Valgrind – segmentation fault

```
1 #include <iostream>
2 using namespace std;
3
4 int main()
5 {
6     int * array1 = new int[1000];
7
8     cout << "hello" << endl;
9     array1[-5] = 5;
10    return 0;
11 }
```

```
==32397== Command: ./testValgrind
==32397==
hello
==32397== Invalid write of size 4
==32397==    at 0x4008D7: main (testValgrind.cpp:9)
==32397== Address 0x5a1c02c is 20 bytes before a block of size 4,000 alloc'd
==32397==    at 0x4C2B800: operator new[](unsigned long) (in /usr/lib/valgrind/vgpreload_memcheck-amd64-linux.so)
==32397==    by 0x4008AE: main (testValgrind.cpp:6)
==32397==
==32397== HEAP SUMMARY:
==32397==    in use at exit: 4,000 bytes in 1 blocks
==32397==    total heap usage: 1 allocs, 0 frees, 4,000 bytes allocated
==32397==
==32397== LEAK SUMMARY:
==32397==    definitely lost: 4,000 bytes in 1 blocks
==32397==    indirectly lost: 0 bytes in 0 blocks
==32397==    possibly lost: 0 bytes in 0 blocks
==32397==    still reachable: 0 bytes in 0 blocks
==32397==    suppressed: 0 bytes in 0 blocks
```

Valgrind – software profiling

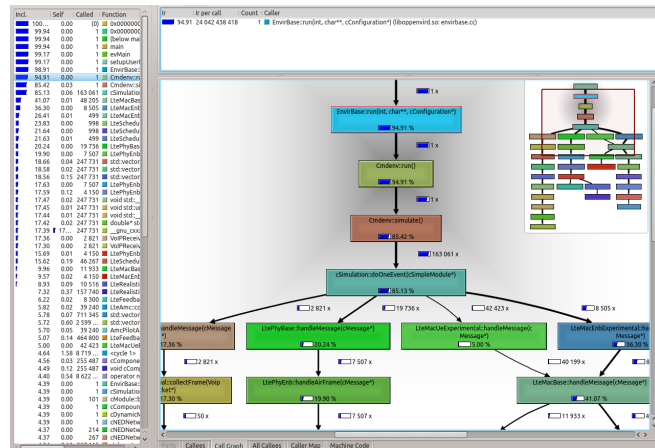
```
$ valgrind --tool=callgrind ./executable
```

• Callgrind

- Call-graph analyzer
- Finds out which functions are called, how many times, and by whom
- Text format output

• Kcachegrind

- Visualize output from Callgrind



The fact that Valgrind checks and counts every instruction in our code, has several *side* benefits. E.g., we can use to do profiling of our code, i.e., evaluate its performance.

Callgrind records the call history of our program

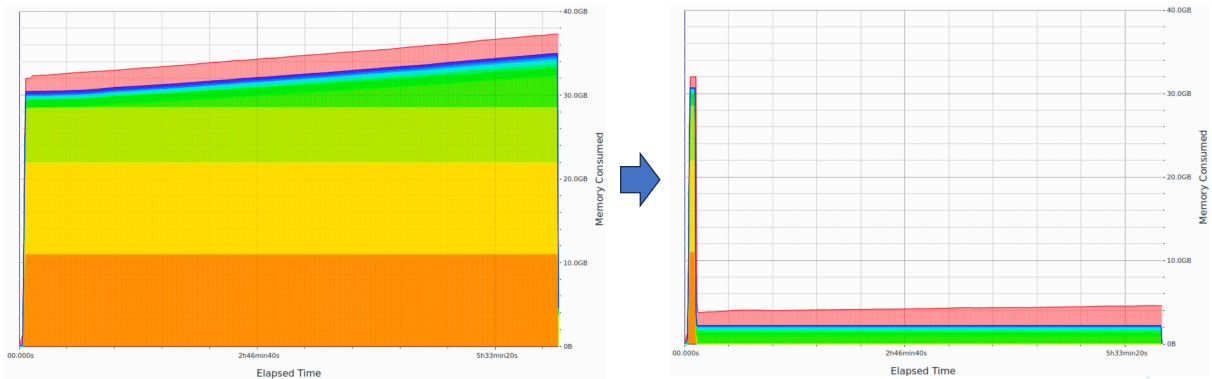
Profiling: a “real-life story”

GN: *run a simulation with 5000 nodes for one hour of simulated time*

Computer: *takes **6 hours** and occupies **~38GB** of RAM*

GN: “Oh no... okay, let’s try and profile the code”

GN: *adds **one** line of code and re-run the same simulation*



The OMNeT++ Simulation Framework

Ing. Giovanni Nardini - University of Pisa - All rights reserved

102

Charts produced with *heaptrack*.

After more work, I was able to eliminate residual leaks and the peak at the initialization so that max peak is at ~3/4 GB.

Execution time fell down to less than ~1h (~11 hours to run 24 hours of simulated time)

There is more...

- Channels
- **Dynamic** modules and connections **creation**
- Python-based result extraction and analysis
- ...
- **INET**

- Can define behaviors (C++ classes) for connections between modules
- Example of mobile nodes in a given geographical area, which enters/leaves the simulated floorplan
- Tool for plotting charts in the OMNeT++ IDE is based on Python (see the manual for details on how to use them).
- Other libraries from OMNeT++ developers and the community, which you can integrate in your project (e.g., INET for all TCP/IP stuff, such as protocols, routers, switches, etc.)